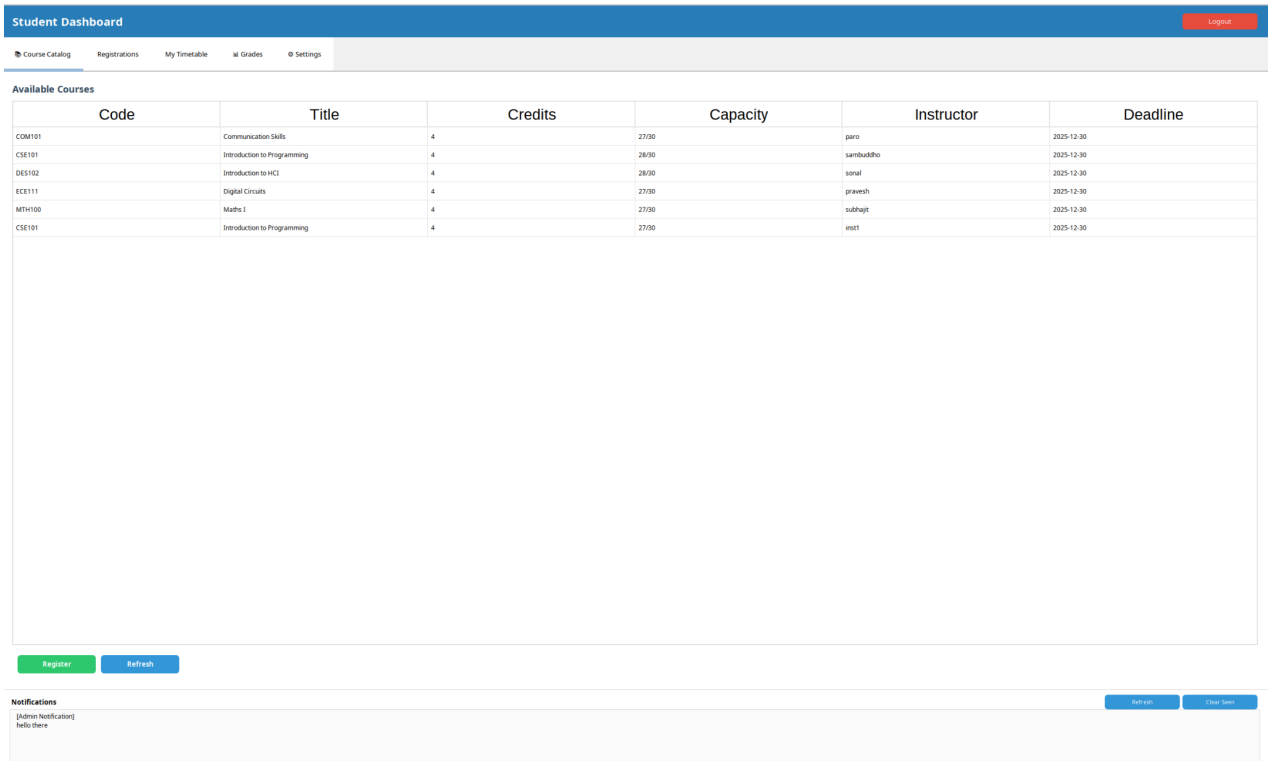
Report and Diagrams

Short report (5–7 pages): screenshots; final-grade weighting rule; how roles & maintenance are enforced; table lists for both DBs; extras added.

Roles & permissions

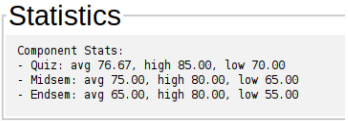
Student (5 tabs)

- **Course Catalog:** View all open courses/sections; **register only if current date ≤** `reg_deadline` (stored on the section).
- **Registrations:** See enrolled sections with instructor and schedule; **drop only if** `reg_deadline` **not passed**.
- **My Timetable:** View personal schedule (read via basic helper retrieval).
- **Grades:** View component scores once instructors post them (instructor writes to DB; student reads via helpers).
- **Settings:** Change password.



Instructor (3 tabs)

- **My Sections:** See assigned sections and enrolled students.



- **Grade Management:** Pick a section → view gradebook (student info + components), **enter marks**, **compute final grades**, **view class stats** (all via `InstructorService` + DB helpers).

- **Settings:** Change password.

Admin (5 tabs)

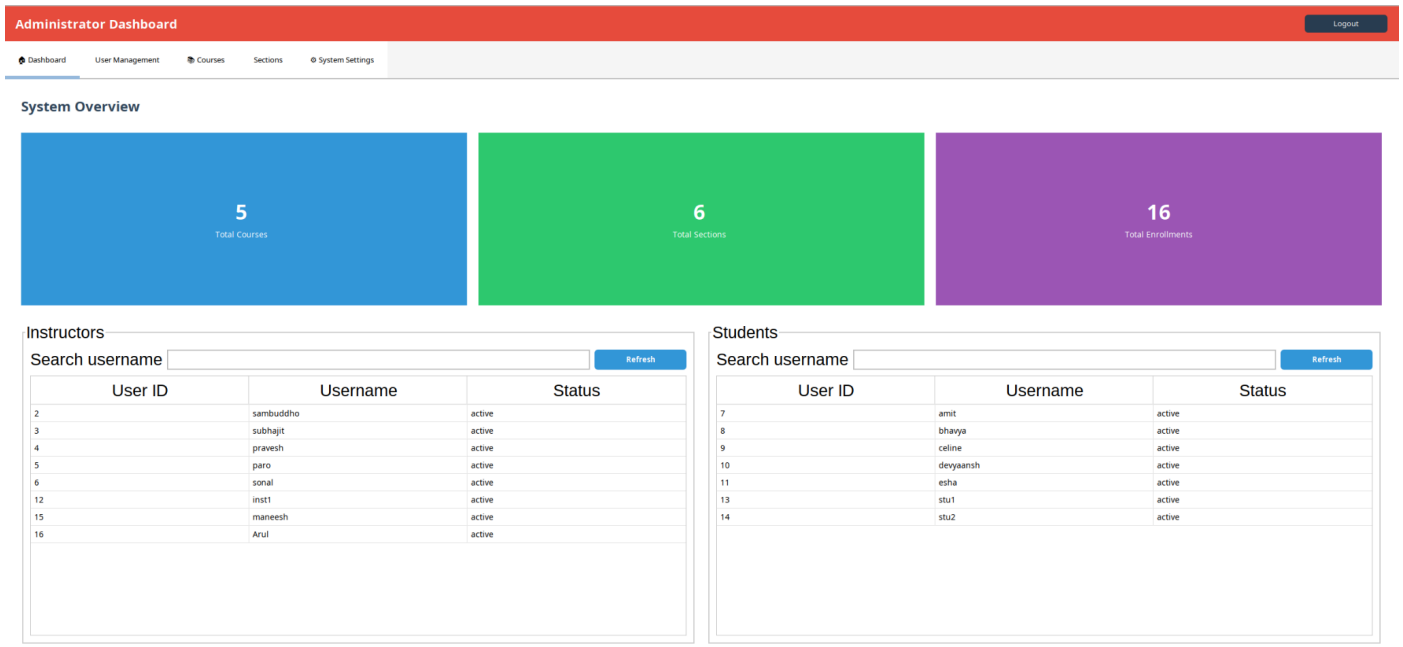
- **Dashboard:** Overview of courses, sections, enrollments.

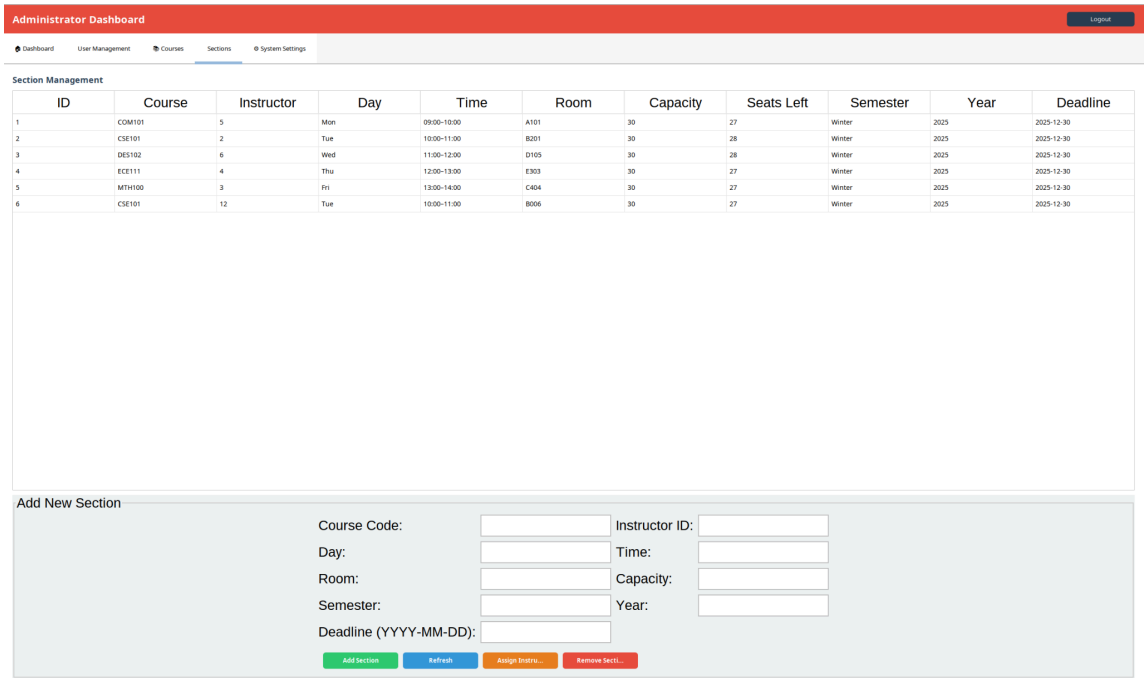
- **User Management:** Create students/instructors/admins; view/manage users; **lock/unlock accounts**.

- **Courses:** View/create courses.

- **Sections:** View/create sections; **assign instructor to section**.

- **Settings:** Change own password; **toggle Maintenance Mode (ON/OFF)**.





Enforcement highlights

- Registration/drop actions are permitted **only before the section's `reg_deadline`**.
- Student grade viewing is **read-only**; instructors are the **sole writers** of grades.
- Admins control **account status** and **maintenance mode**; during maintenance, restrict non-admin write operations as required.

Classes(structs):

The domain folder contains the core classes/structs: **Enrollment, Grade, Setting, Section, User, Student, and Instructor**. I've implemented getters and setters for the necessary fields in each of these classes.

User Data and Login Mechanism

- All login-related information for system users is stored in the `auth_db.users_auth` table (described in the database section below).

- Passwords are hashed using **jBCrypt**, and only the hashes are stored to reduce the risk in case of a database leak.

user_id	username	role	password_hash	status
1	admin1	Admin	$2a$10$7ZaQfqiY259mv7orHPfqQemkxMBU.eEHV4irj56oJZa2.TOWCQ/RC	active

- The **ERP database does not store any passwords**.

- During login, authentication is performed against the **Auth DB**, and once successful, the corresponding **student/instructor profile** is retrieved from the **ERP DB** using the shared **user ID**.

Maintenance Mode and Enforcement

- **Configuration:**
  - Admins can enable or disable maintenance mode from the **Settings** screen.
  - The toggle updates a maintenance flag in the **Settings** table.

- **User Interface:**
  - When maintenance mode is enabled, a **banner(image below)** is displayed at the top of the UI for all users.

- **Behavioral Impact:**
  - All **write operations** from students and instructors are blocked.
  - They can **only read** existing data; no updates or new submissions are allowed.

- **Enforcement Logic:**
  - A helper function `isMaintenanceOn()` is defined.
  - This function is called in shared helper logic to prevent student and instructor actions whenever maintenance mode is active.

Function definition:

```
public boolean isMaintenanceOn() {
        String sql = "SELECT value FROM settings WHERE `key`='maintenance'";
        try (Connection conn = dataSource.getConnection();
                Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(sql)) {
            if (rs.next()) {
                return "on".equalsIgnoreCase(rs.getString("value"));
```

```
            }
        } catch (SQLException e) {
            System.err.println("[SQL ERROR] " + e.getMessage());
        }
        return false;
    }
```

Error Handling and Edge Case Management

- **Exception Handling:**
  - All functions include structured exception handling.
  - Relevant error messages are displayed to the user when failures occur.

- **Edge Case Coverage:**
  - Special edge cases specified in the assignment (e.g., **capacity must be positive**, etc.) are explicitly validated and handled in the code.

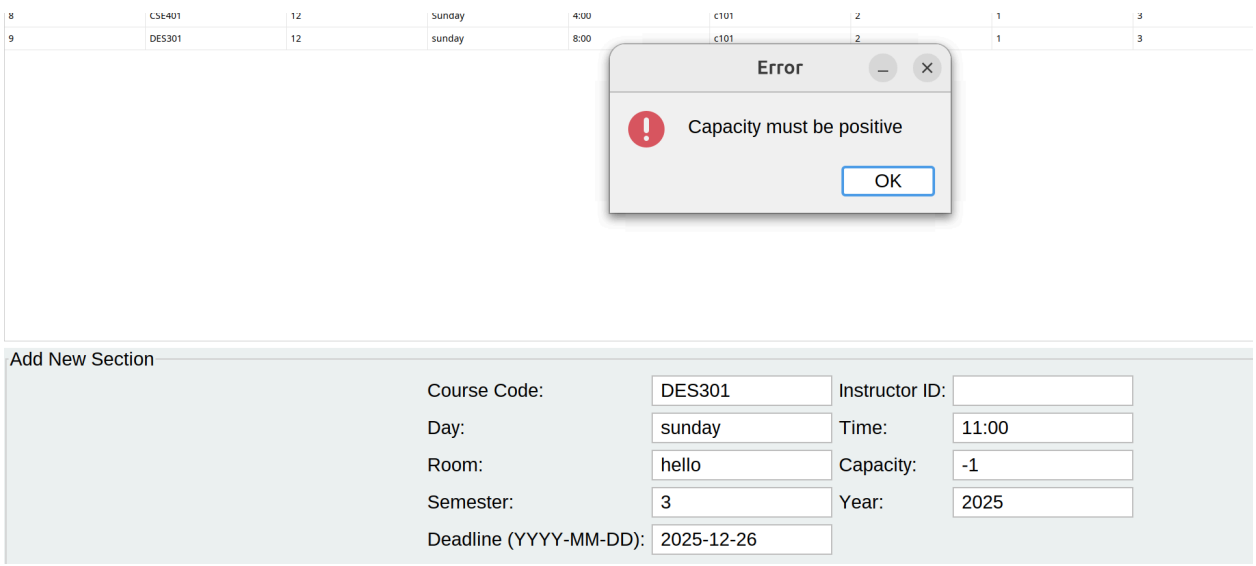


Table Sorting

All tables in the student, instructor, and admin views support column-based sorting: clicking a column header triggers an SQL `ORDER BY` on that field, and the sorted results are returned via helper functions and rendered back in the table

Extra Functionalities

- All tables can be sorted by any field based on the user's choice.
- Every user has a notices window where they receive notices from the administrator.
- Instructors can modify the weightage of assessment components.

Grading and Final Weighting Rule

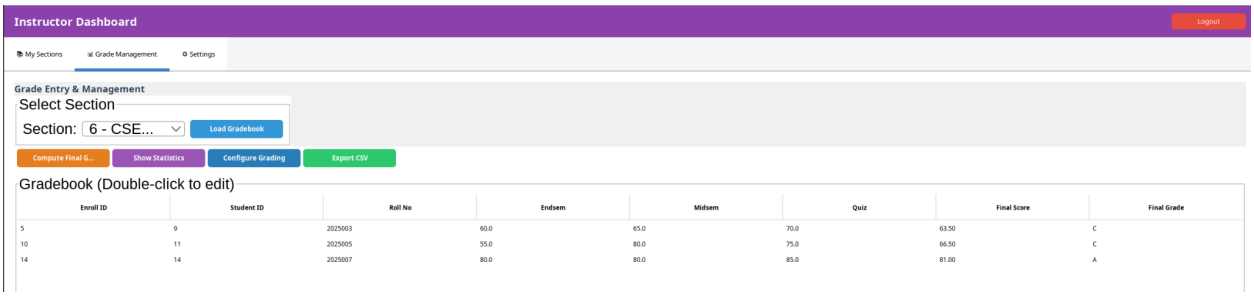
Student View

Students can access a **Grades** tab for their enrolled courses, where they see individual component scores and the final letter grade computed by the course instructor. This is implemented using the `section_weights` table along with helper functions that verify a student's enrollment in a course.

Instructor View

Instructors can:

- Update marks for each assessment component for their students.
- Compute final grades via a dedicated action/button.
- Configure component weightages for the course as needed.
  They can also view **class statistics**, derived from each student's final marks.



Maintenance Mode Interaction

When the administrator enables **maintenance mode**, students and instructors can only **view** grades; no modifications are allowed. This behavior is enforced system-wide by calling a shared `isMaintenanceOn()` function inside all relevant helper functions.

Final Grade Weighting Rule

By default, three components are defined as per the assignment: **Quiz (20%)**, **Midsem (30%)**, and **Endsem (50%)**, each out of **100 marks**. Instructors may adjust these weightages as required. The configured weights are persisted in the `erp_db.section_weights` table for future use.

Database:

We are using HikariCP so instead of creating new database connections for each task, we are maintaining a pool of reusable connections. To make sure the databases are handled separately, we use two separate datasources AuthDataSource and ErpDataSource so that one doesn't run into the other accidentally.

The Database credentials, JDBC URL, and pool settings (environment specific settings) are stored in an external file **application.properties** where fields like db.jdbc.user and db.jdbc.pass can be replaced. The config utility class loads this file at launch and provides a simple interface to the rest of the code.

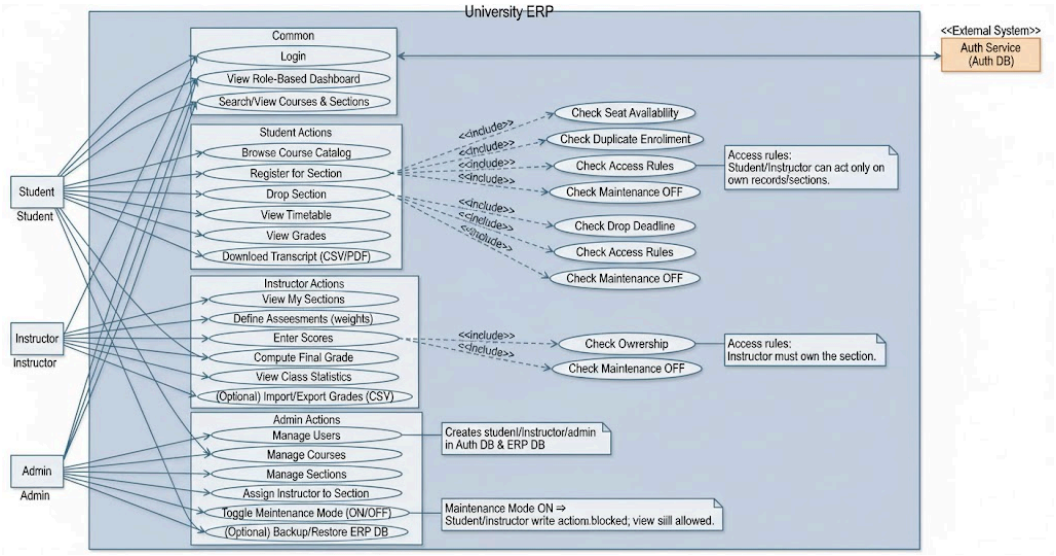
Our project uses MySQL as the primary database. We're following a two-database system separating user authentication data from core ERP(institute and academic data):

- **auth_db** → handles login, user identity, and authorisation. This database is solely dedicated to user identity management. It contains only one table :
    - **users_auth** (user_id, role, username, password_hash, created_at)

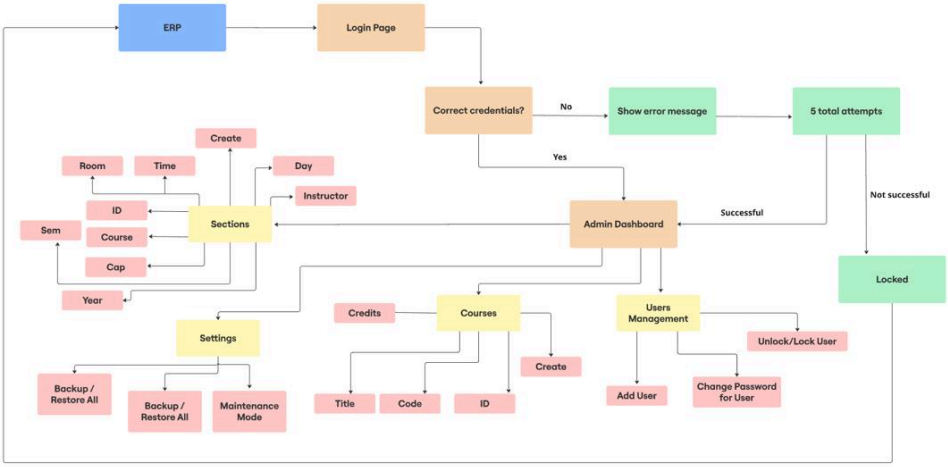
- **erp_db** → stores all functional data such as students, instructors, courses, sections, enrollments, grades etc. This database contains all academic and administrative data. It stores entities and relationships used for :
    - **students** (user_id, roll_no, prog, year)
    - **instructors** (user_id, dept)
    - **courses** (course_id, code, title, credits)
    - **sections** (section_id, course_code, instructor_id, day, time_slot, room, capacity, seats_left, sem, year, reg_deadline)
    - **enrollments** (enroll_id, student_id, section_id, status, enrolled_date)
    - **grades** (grade_id, enrollment_id, component, score, final_grade)
    - **settings** (setting_key, setting_value)
    - **section_weights** (weight_id, section_id, component, weight)

Diagrams

Use Case List:



Admin Workflows:



Student Workflow:

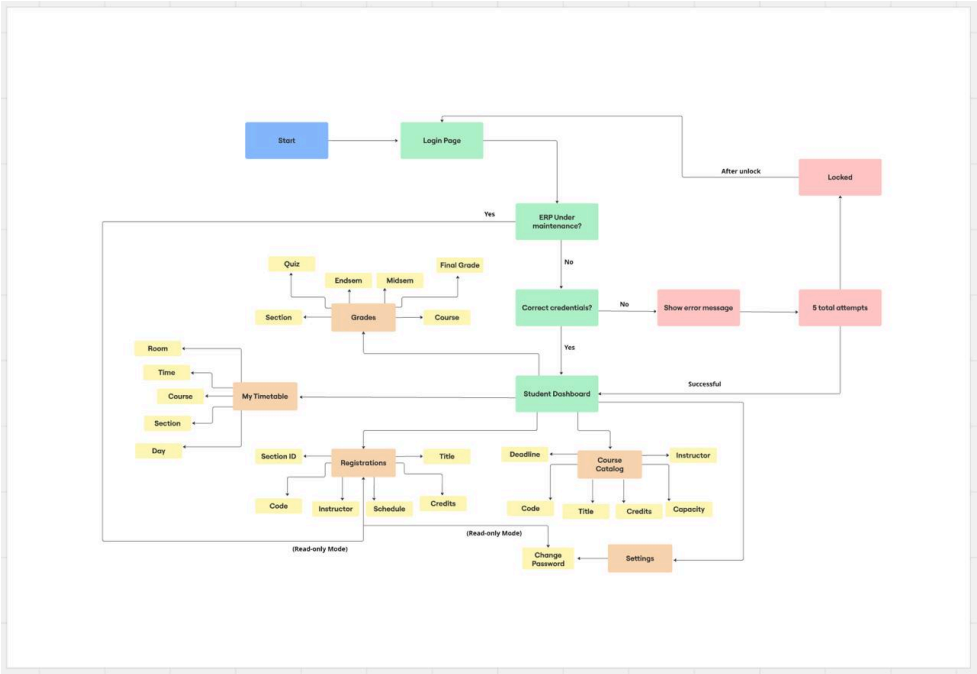
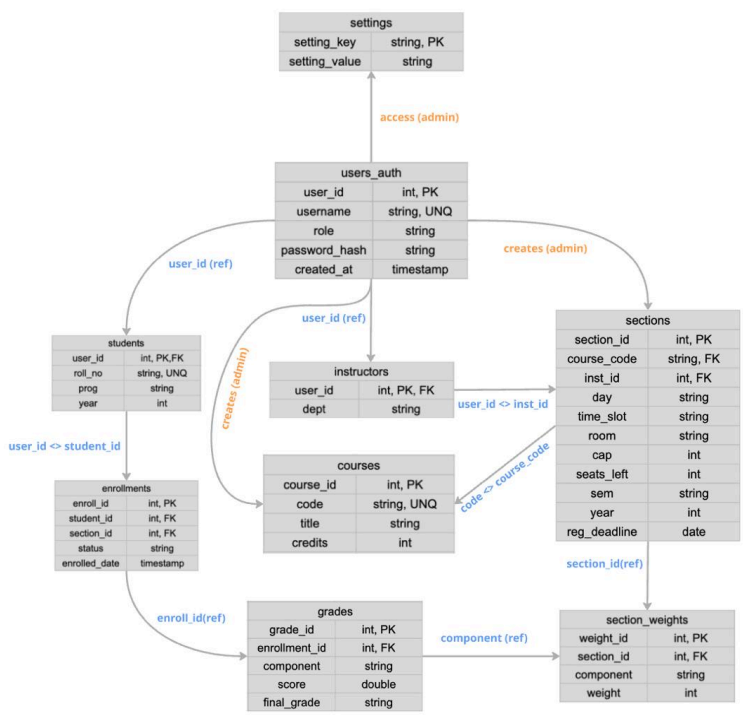


Table Relation Diagram:



Testing pack and How to Run:
All instructions are present in the testing folder attached.(Markdown files).